

IsotopeIQ Satellite

A device-agnostic configuration collection, baseline, and drift-detection platform.

Connect to any managed device, collect its configuration state, store it as a structured canonical JSON baseline, and get alerted when that configuration drifts unexpectedly.

What it does

- **Collects** configuration from Linux, Windows, macOS, network devices, SNMP targets, and any other system reachable via SSH, Telnet, WinRM, HTTPS/API, or the lightweight IsotopeIQ agent
- **Normalises** raw output through parser scripts into a schema-consistent canonical JSON document covering hardware, OS, networking, users, packages, services, firewall rules, certificates, and more
- **Baselines** each device — the last good canonical snapshot is stored per device and updated after every successful collection
- **Detects drift** — every new collection is diffed against the baseline; unexpected changes surface as drift events with a structured side-by-side diff viewer
- **Notifies** via syslog, email (SMTP), and FTP/SFTP export on configurable per-policy triggers
- **Executes arbitrary scripts** on any managed device — remediation, deployment, maintenance, and diagnostic Bundles can be run on a schedule or ad-hoc, independent of the collection/drift workflow
- Supports **LDAP / Active Directory** and **SAML 2.0** single sign-on in addition to local accounts

Technology stack

Component	Technology
Backend API	Django 4 + Django REST Framework
Job queue	Celery + Redis
Scheduler	Celery Beat + <code>django-celery-beat</code>
Database	PostgreSQL 16
Frontend	Vue 3 + Vite + Vuetify 3
SSO	<code>django-auth-ldap</code> , <code>django-saml2</code>

Quick start (Docker)

```
git clone <repository-url> isotopeiq-satellite
cd isotopeiq-satellite

cp .env.example .env
# Edit .env – set SECRET_KEY, FIELD_ENCRYPTION_KEY, DB_PASSWORD, ALLOWED_HOSTS

./deploy.sh up
./deploy.sh createsuperuser
```

Then open `http://<host>:5173` in your browser.

See [ADMIN_GUIDE.md](#) for the full installation, configuration, and operations reference.

Documentation

Document	Audience	Contents
ADMIN_GUIDE.md	System administrators	Installation, Docker setup, environment config, authentication (LDAP/SAML), service management, backup/restore, troubleshooting, security hardening, agent deployment
USER_GUIDE.md	End users	Devices, credentials, scripts, bundles, policies, job monitor, drift review, drift exclusions, baselines, system settings

Core concepts

- Script** — An executable unit (Shell, Python, PowerShell, etc.) that either collects raw data from a device or parses that data into canonical JSON. Four types: Collection, Parser, Deployment, Utility.
- Bundle** — An ordered pipeline of script steps. Steps execute on the remote device (*client*) or on the Satellite server (*server*). Steps can pipe output to each other and flag outputs for baseline storage and drift comparison. Bundles can be exported as `.scriptpack.json` files and shared between Satellite instances.
- Policy** — Binds one or more devices to a Bundle and a cron schedule. The primary automation unit. Supports Agent Pull (no remote script execution) and Script Execution (SSH / Telnet / WinRM / HTTPS) collection methods.
- Baseline** — The most recent successful canonical configuration snapshot for a device. Updated after every successful parse.
- Drift Event** — Created when a new collection differs from the baseline. Includes a structured diff viewer. Resolved by user acknowledgement (which promotes the new state as the baseline) or automatically when the device returns to its previous configuration.
- Drift Exclusion** — A rule that strips expected-volatile fields (uptime, DHCP counts, sysctl entropy values) from diffs before comparison, suppressing false-positive alerts.

Agent — A lightweight daemon (`agents/`) that runs persistently on a managed device, listens on TCP 9322, and responds to `GET /collect` with a canonical JSON snapshot. Useful when SSH/WinRM is unavailable or undesirable.

Canonical JSON schema

All parser scripts must emit a document conforming to the IsotopeIQ canonical schema. The top-level sections are:

```
device hardware os network users groups installed_software
services packages filesystem security listening_services
firewall_rules ssh_authorized_keys ssh_config scheduled_tasks
kernel_modules sysctl certificates vlans acls routing_protocols
spanning_tree port_channels custom
```

Populate inapplicable sections with empty arrays or objects. See the example parsers in [examples/](#) for reference implementations covering Linux, Windows, macOS, Cisco IOS, HP-UX, OpenVMS, ESXi, and SNMP targets.

Example scripts

The [examples/](#) directory contains ready-to-use collector and parser pairs:

Script pair	Target
<code>linux_baseline_*</code>	Linux (SSH or agent)
<code>windows_baseline_*</code>	Windows (WinRM or PowerShell)
<code>macos_baseline_*</code>	macOS (SSH)
<code>cisco_ios_baseline_*</code>	Cisco IOS (SSH)
<code>hpux_baseline_*</code>	HP-UX (SSH)
<code>openvms_baseline_*</code>	OpenVMS (SSH)
<code>esxi_baseline_*</code>	VMware ESXi
<code>snmp_baseline_*</code>	Any SNMP-capable device
<code>passthrough_parser.py</code>	Devices that emit canonical JSON directly

Agent binaries

Pre-compiled agent binaries are in [agents/](#) :

File	Platform
linux_collector_amd64	Linux x86-64 (glibc 2.17+)
linux_collector_i686	Linux x86 32-bit
macos_collector	macOS
windows_collector.exe	Windows 7 32-bit and later

Installers are in [agents/installers/](#). For online devices the agent can be downloaded directly from the UI (**Configuration** → **Agent Download**). For air-gapped devices see [ADMIN_GUIDE.md § 15](#).